PortPilot - scanner TCP de laboratoire

Durée : 5 heures · Niveau débutant · 3 merge requests

Tu vas écrire un petit programme Python qui essaie une connexion TCP vers quelques ports de 127.0.0.1, conserve un état par port et affiche un bilan. Il n'envoie aucune donnée applicative et ne récupère aucune bannière. Un port ouvert signifie uniquement que la connexion TCP a réussi : cela ne prouve ni une vulnérabilité ni la nature du service.

Le parcours complet et les définitions détaillées des fonctions et des huit tests sont dans le PDF PortPilot. Ce fichier reprend aussi les étapes GitLab et les critères de livraison.

Préparation

Utilise le dépôt autorisé par ton école, son URL et sa branche principale (main ou master). Il te faut Python 3, Git, un runner GitLab actif utilisable par les branches de MR et les droits nécessaires aux règles de fusion. Sans runner ou accès aux règles, tu peux développer et exécuter les tests localement, mais la CI bloquante reste non validée.

Le dossier de cours dans Cyber Atelier est vide. Clone le dépôt de l'école directement dans ce dossier avant de créer tes fichiers, avec git clone <URL_DU_DEPOT> . (le point signifie « ici »). Ainsi, le dépôt ne se retrouve pas dans un sous-dossier. Le terminal web lance chaque commande dans le dossier courant explicite et ne garde pas un cd d'une commande à la suivante. Pour une session Git interactive, ouvre un terminal de ta machine et lance docker compose exec atelier bash depuis le dossier de Cyber Atelier. Initialise le dépôt avec README.md et .gitignore ; ce dernier exclut les caches Python et portpilot.zip. Protège la branche principale selon les règles de l'école : pas de push direct, force push désactivé, fusion après pipeline réussi et option pipeline ignoré comme réussi désactivée. Vérifie que le runner accepte les jobs de branches non protégées et les tags demandés.

Contrats Python

Écris cinq fonctions dans portpilot.py et les tests dans verification.py.

- valider_ports(ports) accepte une liste non vide de 1 à 10 ports distincts, entiers de 1 à 65535 inclus. Elle retourne les mêmes valeurs dans le même ordre. Elle lève ValueError pour une liste vide, plus de dix éléments, un doublon, une valeur hors bornes, un texte, un nombre décimal ou un booléen. Ne convertis pas le texte en entier. Une entrée qui n'est pas une liste est hors contrat.
- tester_port(port) utilise uniquement l'hôte constant 127.0.0.1 et le délai fixe de 0,5 seconde. Elle retourne exactement OUVERT si la connexion réussit, REFUSE pour ConnectionRefusedError, TIMEOUT pour un dépassement du délai et ERREUR pour un autre OSError. Toute socket créée est fermée même en cas d'échec.
- scanner_ports(ports) valide toute la liste avant toute connexion, fait une tentative par port dans l'ordre reçu et retourne une liste de paires [port, état] dans ce même ordre.
- resumer(resultats) retourne cinq compteurs entiers : total, ouverts, refuses, timeouts, erreurs. Les catégories totalisent total. Une liste vide donne cinq zéros.
- main coordonne la liste configurée, le scan, l'affichage d'un résultat par port et le bilan. Pour une liste invalide, affiche un message clair. L'import de portpilot par verification.py ne doit pas lancer main.

Reste dans le périmètre : pas de saisie interactive, d'arguments, de classes, de threads, de paquets pip, de plage de ports, de scan externe, d'exploitation, de lecture de fichiers ou de rapport écrit sur disque.

Comment travailler un exercice

Ces exercices sont ton entraînement. Chaque palier explique les notions utiles, montre les données ou les règles attendues, puis te fait écrire et vérifier ton propre code. Les durées sont des repères : avance quand tu comprends le résultat.

1. Lis « Pour comprendre » et l'exemple lorsqu'il y en a un. Repère ce qui entre dans le programme, ce qui doit en sortir et les règles à respecter.
2. Les cartes de tests donnent l'entrée et le résultat attendu. Écris les tests correspondants dans le fichier indiqué ; cocher un palier ne les exécute pas. Avec unittest, « Ran 0 tests » signifie qu'aucun test n'a été exécuté : ce n'est pas une validation. Un cas décrit peut contenir plusieurs valeurs ; le nombre de méthodes Python n'est donc pas forcément le nombre d'identifiants.
3. Suis les actions numérotées, une à la fois. Les exemples servent à apprendre une notion ; tu écris toi-même la solution de l'exercice. Les blocs Terminal donnent les commandes à lancer au moment indiqué.
4. Vérifie le résultat indiqué avant de poursuivre. Un échec attendu (phase rouge, argument refusé ou conflit volontaire) est expliqué dans le palier.
5. Si tu bloques, lis la dernière ligne de l'erreur, compare l'entrée et l'attendu, puis ouvre l'indice. Fais un petit changement et relance.

Le bon endroit pour chaque action

Sur ton Mac, ouvre `workspaces/portpilot` dans VS Code. Dans Cyber Atelier, utilise l'éditeur et la console de **Coder**, avec `cwd` sur `.` (la racine du dépôt). Fais la préparation de **Environnement** avant MR1 ; la mission crée ensuite sa branche.

Le code Python se place dans l'éditeur. Les commandes `docker compose ...` se lancent sur ton Mac depuis `cyberatelier` ; `/app/...` existe dans le conteneur. Le laboratoire interrogé doit tourner dans le même environnement que ton programme.

La console web repart à neuf à chaque exécution : un `cd`, une activation de venv ou `echo $?` ne se transmet pas à la commande suivante. Lance le programme et `echo $?` ensemble quand le palier le demande. Le nombre affiché par echo est celui du programme ; la console peut afficher code 0 pour echo lui-même.

Un venv créé sur le Mac ne fonctionne pas dans le conteneur : utilise `python3` dans ce dernier. Remplace `main` par `master` si nécessaire. Si une branche existe déjà, reprends-la avec `git switch NOM` sans `-c`. Après chaque fusion, actualise la branche principale avant la mission suivante.

Les mots utiles

Chaîne / champ. Une chaîne (str) est du texte. Un champ est un morceau de donnée, comme l'utilisateur dans une ligne de journal.

Liste / position. Une liste conserve plusieurs valeurs dans un ordre. Sa première position porte le numéro 0, la suivante 1, etc.

Dictionnaire / clé. Un dictionnaire associe un nom à une valeur : {"fruit": "pomme"}. fruit est la clé ; pomme est sa valeur.

None. Une valeur Python qui représente l'absence de résultat. Dans analyser_ligne, elle signifie que la ligne est refusée. Elle s'écrit sans guillemets.

split / strip. split() sans argument découpe une chaîne sur ses espaces. split(".") découpe sur les points. strip() retire les espaces aux bords.

Contrat. Les règles à respecter : arguments reçus, valeur retournée, erreurs et cas limites. Tu choisis le code qui respecte ces règles.

Test à écrire. Une entrée et un résultat attendu à transformer en vérification Python. Les identifiants S1, T1, etc. désignent des cas à couvrir, pas des tests déjà installés.

Rouge puis vert. Écris la vérification, lance-la et comprends son échec. Écris ensuite le code, relance et garde les vérifications précédentes. Un nouveau cas peut déjà passer : ne casse pas le code pour le rendre rouge.

Fonction pure. Une fonction qui calcule son résultat depuis ses arguments, sans accéder au réseau ou au disque et sans modifier les données reçues.

Exception. Une erreur signalée par Python, par exemple ValueError. « Lever » signale l'erreur ; « attraper » la traite avec except. Si le contrat attend une exception, la voir est une réussite du test.

Fixture. Un fichier de données fictives fourni pour rendre un essai reproductible. Une fixture XML ou JSON remplace la sortie d'un outil réel dans les tests.

Mock. Un faux temporaire qui remplace un appel pendant le test. Il permet de choisir le résultat et de vérifier les arguments sans lancer de réseau ou de processus réel.

CLI. L'interface en ligne de commande : tu lances un programme avec des arguments. --help décrit les options ; argparse les lit et vérifie leur forme.

Code de sortie. Le nombre rendu au terminal quand le programme se termine. 0 signifie réussite ; une valeur non nulle signale un échec ou un usage incorrect selon le contrat.

Commit. Un point enregistré dans l'historique Git. git add choisit les fichiers ; git diff --staged montre le contenu choisi ; git commit l'enregistre.

PR / MR. Une demande de fusion de ta branche vers la principale. GitHub dit pull request (PR), GitLab merge request (MR). L'ouvrir, la relire et la fusionner sont trois actions distinctes.

Pipeline / job. GitLab exécute le pipeline : un ensemble de tâches, appelées jobs. La CI est cette exécution automatique. Un fichier de configuration présent ne prouve pas qu'un job a tourné.

Artefact. Un fichier conservé par un job GitLab après son exécution, par exemple le ZIP de livraison.

Empreinte / manifeste. L'empreinte SHA-256 est calculée à partir des octets d'un fichier. Le manifeste mémorise des noms et leurs empreintes pour comparer plus tard.

Parser. Lire un format structuré pour obtenir des données Python. JSON donne des dictionnaires et listes ; XML donne un arbre de balises et d'attributs.

Package / venv. Un package regroupe des modules Python dans un dossier. Un venv est un environnement Python séparé, que tu ne livres pas dans Git.

Déterministe. Avec les mêmes entrées, le résultat est identique. Pour tester un rapport daté, le test fournit la date au lieu de lire l'heure actuelle.

stderr / stdout. Deux sorties du programme : stdout porte le résultat normal, stderr les messages d'erreur et les traces. Le code de sortie reste un nombre séparé.

Loopback. L'adresse de la machine où tourne le programme. 127.0.0.1 dans Docker désigne le conteneur ; sur le Mac, elle désigne le Mac ; dans la CI, elle désigne le runner.

Les trois MR

Les durées sont des repères compris dans les 300 minutes du parcours.

MR1 - feat/validation (80 min)

Tu prouves aussi qu'une régression rend le pipeline rouge et bloque la fusion. Avance dans l'ordre des paliers et vérifie chaque résultat avant de poursuivre.

Avant de commencer

- Point de départ : Un dépôt GitLab cloné, initialisé avec README.md et .gitignore ; l'accès au runner et aux règles de fusion est vérifié.
- Ce que tu vas produire : portpilot.py et verification.py : valider la liste de ports ; .gitlab-ci.yml : exécuter les tests de MR.
- Pourquoi : Tu prouves aussi qu'une régression rend le pipeline rouge et bloque la fusion.
- Commande des tests depuis la racine : `python3 verification.py`. Les cas ci-dessous sont à écrire par toi.

Palier 1 - Partir d'une branche à jour (10 min)

Créer la branche de la MR depuis une branche principale à jour, sans rien emporter d'inutile.

Où tu travailles : Dépôt

Pour comprendre

- La branche principale (main) contient le travail déjà fusionné. Ta branche feat/... reçoit les changements de cette mission.
- Un dépôt « propre » n'a aucune modification en attente : git status te le confirme.

Actions dans l'ordre

1. Ouvre workspaces/portpilot dans VS Code, ou Coder dans Cyber Atelier avec cwd sur « . ».
2. Termine la préparation de l'onglet Environnement avant de commencer.
3. Lis git status ; enregistre ton travail en cours avant de changer de branche.
4. Reviens sur main avec git switch main, mets-la à jour avec git pull --ff-only, puis crée feat/validation. Remplace main par master si ton dépôt utilise ce nom.
5. Vérifie le nom de ta branche avec git status. Si elle existe déjà, reprends-la avec git switch sans -c.

Terminal · Relire l'état du dépôt

Terminal du parcours : reste à la racine du dépôt. Exécute ces lignes au moment demandé dans les actions, avec les modifications manuelles entre les blocs.

```
git status
```

Terminal · Après la fusion : actualiser la principale

```
git switch main
```

Terminal · Commande à lancer au moment indiqué

```
git pull --ff-only
```

Terminal · Commande à lancer au moment indiqué

```
git switch -c feat/validation
```

Terminal · Relire l'état du dépôt

```
git status
```

Tu sais que c'est bon quand : git status affiche « On branch feat/validation » et « nothing to commit, working tree clean ».

Indice si tu bloques : Si git pull --ff-only refuse, c'est que ta branche principale locale a des commits qui ne viennent pas du dépôt de l'école : ne force rien, demande de l'aide avant de continuer. Si git status signale des fichiers modifiés, lis-les avec git diff avant de décider quoi faire.

Palier 2 - Premier test rouge : T1 (10 min)

Écrire un test qui appelle valider_ports, le voir échouer, et comprendre ce qui fait échouer un test.

Où tu travailles : verification.py

Pour comprendre

- assert vérifie une condition : si elle est fausse, le test échoue avec AssertionError. Le message de succès doit venir après la vérification.
- Définir une fonction test_t1 ne l'exécute pas : verification.py doit aussi l'appeler.

Éditeur · Exemple Python à lire ou essayer

Éditeur : copie cet exemple dans exemple.py (jetable), enregistre-le, puis lance python3 exemple.py. Si plusieurs exemples sont proposés, essaie-les un par un en remplaçant le contenu du fichier. Retire ensuite les fichiers générés et le fichier d'exemple.

```
def doubler(n):
    return n * 2

assert doubler(4) == 8, "doubler(4) doit valoir 8"
print("la condition est vraie, rien ne se passe")
```

Cas de test à écrire

L'entrée est ce que tu donnes au code ; l'attendu est ce que ton test doit vérifier.

T1 - Liste valide et ordre conservé

- Entrée : [1, 8000, 65535]
- Résultat attendu : La même liste, dans le même ordre, est acceptée.

Actions dans l'ordre

1. Crée verification.py à la racine, importe portpilot et écris test_t1.
2. T1 appelle valider_ports([1, 8000, 65535]) et compare le retour à cette liste, dans le même ordre.
3. Appelle test_t1 à la fin du fichier, puis lance verification.py.
4. Lis l'échec attendu : portpilot.py ou valider_ports n'existe pas encore. Ne commite pas ce test seul avant le code du prochain palier.

Terminal · Commande à lancer au moment indiqué

Terminal du parcours : reste à la racine du dépôt. Exécute ces lignes au moment demandé dans les actions, avec les modifications manuelles entre les blocs.

```
python3 verification.py
```

Tu sais que c'est bon quand : python3 verification.py se termine par une erreur (ModuleNotFoundError ou AssertionError). Tu sais dire pourquoi.

Indice si tu bloques : Lis la dernière ligne du message d'erreur : elle indique le module manquant ou la valeur qui ne correspond pas. Pour l'exemple doubler, remplace 8 par 9 si tu veux voir un assert échouer, puis remets 8.

Palier 3 - T1 vert : le minimum de code (10 min)

Écrire le minimum de code qui fait passer T1, puis faire ton premier commit de code.

Où tu travailles : portpilot.py ; verification.py

Pour comprendre

- Valider une liste signifie décider si toute son entrée respecte les règles. Une liste admise est retournée avec les mêmes valeurs et le même ordre.
- Tu complètes progressivement les règles ; la fonction finale devra toutes les respecter.

Exemple de résultat attendu

Ce que tu donnes :

```
valider_ports([1, 8000, 65535])
```

Ce que tu dois obtenir :

```
[1, 8000, 65535]
```

La validation ne trie pas la liste et ne transforme pas les éléments.

Règles à respecter

- De 1 à 10 ports, sans doublon.
- Chaque port est un entier de 1 à 65535 inclus ; les booléens sont refusés.
- Une liste invalide lève ValueError. Une liste admise conserve valeurs et ordre.

Actions dans l'ordre

1. Crée portpilot.py et valider_ports(ports) pour accepter T1.
2. Relance verification.py et vérifie que test_t1 a réellement été appelé.
3. Vérifie que .gitignore exclut __pycache__/ ; relis status et le diff.
4. Commite portpilot.py et verification.py quand T1 passe.

Terminal · Commande à lancer au moment indiqué

Terminal du parcours : reste à la racine du dépôt. Exécute ces lignes au moment demandé dans les actions, avec les modifications manuelles entre les blocs.

```
python3 verification.py
```

Terminal · Relire l'état du dépôt

```
git status
```

Terminal · Enregistrer le travail vérifié

```
git add portpilot.py verification.py
git commit -m "Ajoute valider_ports et le test T1"
```

Terminal · Relire l'état du dépôt

```
git log --oneline
```

Tu sais que c'est bon quand : verification.py affiche T1 comme réussi, et git log --oneline montre ton commit « Ajoute valider_ports et le test T1 ».

Indice si tu bloques : Le minimum de code n'est pas la solution finale : les règles suivantes arriveront palier après palier, chacune avec son test. Un commit par palier de code garde l'historique lisible.

Palier 4 - Code de sortie et T2 : la liste vide (10 min)

Comprendre pourquoi un processus doit rendre un code non nul en cas d'échec, puis écrire T2.

Où tu travailles : portpilot.py ; verification.py

Pour comprendre

- Le code de sortie indique au terminal si le programme a réussi : 0 pour succès, une valeur non nulle pour échec. sys.exit transmet ce nombre.
- Un test d'exception ne réussit que si la bonne exception est levée. Aucun refus ou une autre erreur doivent compter comme un échec.

Éditeur · Exemple Python à lire ou essayer

Éditeur : copie cet exemple dans exemple.py (jetable), enregistre-le, puis lance python3 exemple.py. Si plusieurs exemples sont proposés, essaie-les un par un en remplaçant le contenu du fichier. Retire ensuite les fichiers générés et le fichier d'exemple.

```
import sys

print("un test passe")
print("un autre échoue")
sys.exit(1)
```

Éditeur · Exemple Python à lire ou essayer

```
def diviser(a, b):
    if b == 0:
        raise ValueError("diviseur nul")
    return a / b

try:
    diviser(1, 0)
except ValueError:
    print("attendu : ValueError levée")
```

Cas de test à écrire

L'entrée est ce que tu donnes au code ; l'attendu est ce que ton test doit vérifier.

T2 - Liste vide

- Entrée : []
- Résultat attendu : ValueError.

Actions dans l'ordre

1. Essaie l'exemple sys.exit, puis lis echo $? dans la même exécution que le programme.
2. Ajoute T2 : valider_ports([]) doit lever ValueError.
3. Dans verification.py, appelle chaque test, compte les échecs et rends un code non nul dès qu'un test échoue. Ne masque pas les exceptions inattendues.
4. Complète valider_ports pour refuser la liste vide ; vérifie T1 et T2 ensemble.
5. Change temporairement un attendu pour vérifier le code non nul, puis restaure l'attendu correct avant de commiter.

Terminal · Commande à lancer au moment indiqué

Terminal du parcours : reste à la racine du dépôt. Exécute ces lignes au moment demandé dans les actions, avec les modifications manuelles entre les blocs. Lance le programme et echo $? ensemble : le nombre affiché est le code du programme ; la console peut afficher code 0 pour echo lui-même.

```
python3 exemple.py; echo $?
```

Terminal · Commande à lancer au moment indiqué

```
python3 verification.py; echo $?
```

Terminal · Enregistrer le travail vérifié

Terminal du parcours : reste à la racine du dépôt. Exécute ces lignes au moment demandé dans les actions, avec les modifications manuelles entre les blocs.

```
git add portpilot.py verification.py
git commit -m "Rejette la liste vide dans valider_ports"
```

Tu sais que c'est bon quand : python3 verification.py; echo $? affiche 0 quand tout passe et une valeur non nulle quand une assertion est ratée. T2 passe.

Indice si tu bloques : Pour savoir si une exception a bien été levée, un bloc try avec except ValueError ne suffit pas : il faut aussi traiter le cas où rien n'est levé. Le mot-clé else d'un try peut t'aider. Ne mets pas python -O : il désactive les assert.

Palier 5 - T3 et T5 : onze ports, doublon (10 min)

Ajouter deux règles simples (limite de nombre et unicité), chacune avec son test.

Où tu travailles : portpilot.py ; verification.py

Pour comprendre

- len(liste) compte les éléments. set(liste) crée un ensemble qui ne garde qu'une occurrence de chaque valeur.
- Le contrat refuse les doublons ; il ne demande pas de les supprimer silencieusement.

Éditeur · Exemple Python à lire ou essayer

Éditeur : copie cet exemple dans exemple.py (jetable), enregistre-le, puis lance python3 exemple.py. Si plusieurs exemples sont proposés, essaie-les un par un en remplaçant le contenu du fichier. Retire ensuite les fichiers générés et le fichier d'exemple.

```
fruits = ["pomme", "poire", "pomme"]
print(len(fruits))
print(len(set(fruits)))
```

Cas de test à écrire

L'entrée est ce que tu donnes au code ; l'attendu est ce que ton test doit vérifier.

T3 - Onze ports distincts

- Entrée : Onze entiers distincts valides
- Résultat attendu : ValueError.

T5 - Doublon

- Entrée : [8000, 8000]
- Résultat attendu : ValueError.

Actions dans l'ordre

1. Observe l'exemple des fruits avec une valeur répétée : compare les deux longueurs.
2. Ajoute T3 avec onze ports distincts valides : ValueError attendue.
3. Ajoute T5 avec [8000, 8000] : ValueError attendue.
4. Complète les contrôles de taille et de doublon, relance après chaque changement.
5. Vérifie T1, T2, T3 et T5 puis commite.

Terminal · Commande à lancer au moment indiqué

Terminal du parcours : reste à la racine du dépôt. Exécute ces lignes au moment demandé dans les actions, avec les modifications manuelles entre les blocs.

```
python3 verification.py
```

Terminal · Enregistrer le travail vérifié

```
git add portpilot.py verification.py
git commit -m "Limite à dix ports et refuse les doublons"
```

Tu sais que c'est bon quand : T1, T2, T3 et T5 passent, et python3 verification.py se termine avec le code 0.

Indice si tu bloques : Compare la longueur de la liste à ce qui est autorisé avant de tester les doublons. Pour les doublons, compare le nombre d'éléments de la liste avec celui de la version en ensemble : c'est une idée, pas la fonction finale.

Palier 6 - T4 : six valeurs invalides, testées séparément (10 min)

Tester chaque valeur invalide dans sa propre liste, et comprendre pourquoi bool passe pour un entier.

Où tu travailles : portpilot.py ; verification.py

Pour comprendre

- Python considère True comme un int avec isinstance : il faut donc refuser explicitement les booléens avant d'accepter les entiers.
- Une boucle de tests doit rapporter chaque cas ; le premier échec ne doit pas cacher les cinq autres.

Éditeur · Exemple Python à lire ou essayer

Éditeur : copie cet exemple dans exemple.py (jetable), enregistre-le, puis lance python3 exemple.py. Si plusieurs exemples sont proposés, essaie-les un par un en remplaçant le contenu du fichier. Retire ensuite les fichiers générés et le fichier d'exemple.

```
print(isinstance(3, int))
print(isinstance(True, int))
print(isinstance("3", int))
```

Cas de test à écrire

L'entrée est ce que tu donnes au code ; l'attendu est ce que ton test doit vérifier.

T4 - Valeurs invalides

- Entrée : -1, 0, 65536, texte, décimal et booléen, testés séparément
- Résultat attendu : ValueError pour chacun ; aucune autre exception ne passe.

Actions dans l'ordre

1. Essaie les trois appels isinstance de l'exemple.
2. Écris les six sous-cas de T4 : [-1], [0], [65536], ["8000"], [80.5] et [True].
3. Vérifie ValueError pour chacun, avec un nom de cas affiché si le contrôle échoue.
4. Complète le contrôle du type et des bornes ; relance T1 à T5 avec les six sous-cas.
5. Commite lorsque la suite rend 0.

Terminal · Commande à lancer au moment indiqué

Terminal du parcours : reste à la racine du dépôt. Exécute ces lignes au moment demandé dans les actions, avec les modifications manuelles entre les blocs.

```
python3 verification.py
```

Terminal · Enregistrer le travail vérifié

```
git add portpilot.py verification.py
git commit -m "Refuse les valeurs invalides, booléens compris"
```

Tu sais que c'est bon quand : Les six sous-cas de T4 passent séparément ; le nom d'un cas en échec serait affiché.

Indice si tu bloques : Un booléen est une sorte d'entier pour Python. Refuse d'abord True et False avant d'accepter un entier. Pour le décimal, écris bien 80.5 en Python : 80,5 seul serait une liste de deux entiers.

Palier 7 - CI : fichier, MR et premier pipeline vert (10 min)

Relier la même commande à GitLab : un job qui n'exécute que sur les merge requests, puis une première MR verte.

Où tu travailles : .gitlab-ci.yml ; MR GitLab

Pour comprendre

- GitLab lit .gitlab-ci.yml pour exécuter des jobs sur son runner. Un pipeline de MR est déclenché pour une demande de fusion.
- Un job bloquant fait échouer le pipeline quand sa commande échoue. Le lint vérifie seulement la configuration YAML.

Actions dans l'ordre

1. Crée .gitlab-ci.yml à la racine et rédige sa configuration toi-même.
2. Limite les pipelines à merge_request_event ; un simple push sans MR ne doit pas déclencher le pipeline.
3. Ajoute le job verifier avec python verification.py, ou python3 selon l'environnement.
4. Rends-le bloquant : pas de allow_failure, pas de job manuel, pas de règle finale qui accepte tous les événements.
5. Lance la même commande en local, puis commite et pousse la CI. Ouvre la MR non Draft vers main.
6. Dans GitLab, vérifie que verifier a réellement tourné et réussi sur le dernier commit.

Terminal · Enregistrer le travail vérifié

Terminal du parcours : reste à la racine du dépôt. Exécute ces lignes au moment demandé dans les actions, avec les modifications manuelles entre les blocs.

```
git add .gitlab-ci.yml
git commit -m "Ajoute la CI de merge request avec le job verifier"
```

Terminal · Publier la branche ou ouvrir la demande de fusion

```
git push -u origin feat/validation
```

Terminal · Relire l'état du dépôt

```
git log --oneline -1
```

Tu sais que c'est bon quand : La MR existe, son pipeline tourne sur le dernier commit et le job verifier est vert. Ce vert est un résultat de pipeline, pas un test local.

Indice si tu bloques : Si aucun pipeline ne se lance, vérifie la règle merge_request_event et que la MR est bien ouverte depuis feat/validation. Un test local réussi ne suffit pas : ce qui compte pour la fusion est le pipeline du dernier commit de la MR.

Palier 8 - Régression, rouge bloquant, correction et fusion (10 min)

Observer qu'un pipeline rouge bloque la fusion, corriger le code sans toucher aux tests, puis fusionner le dernier commit vert.

Où tu travailles : portpilot.py ; MR GitLab

Pour comprendre

- Une régression est une modification qui casse un comportement déjà vérifié. Ici, tu la provoques pour voir la CI bloquer la fusion.
- Après une correction, seul le pipeline du dernier commit décrit l'état à fusionner ; un ancien vert ne suffit pas.

Actions dans l'ordre

1. Retire volontairement une règle de portpilot.py, sans modifier les tests.
2. Commite et pousse cette régression ; attends le pipeline rouge et relève son lien, le test en échec et son message.
3. Observe le blocage de la fusion selon la règle de l'école.
4. Corrige le code, commite et pousse ; vérifie le nouveau pipeline vert du dernier commit.
5. Relis le diff, fusionne lorsque les règles le permettent, puis mets main à jour.
6. Conserve le lien du pipeline rouge pour le bilan.

Terminal · Relire l'état du dépôt

Terminal du parcours : reste à la racine du dépôt. Exécute ces lignes au moment demandé dans les actions, avec les modifications manuelles entre les blocs.

```
git diff
```

Terminal · Enregistrer le travail vérifié

```
git add portpilot.py
git commit -m "Introduit une régression volontaire pour tester le blocage"
```

Terminal · Publier la branche ou ouvrir la demande de fusion

```
git push
```

Terminal · Enregistrer le travail vérifié

```
git add portpilot.py
git commit -m "Corrige la régression de valider_ports"
```

Terminal · Publier la branche ou ouvrir la demande de fusion

```
git push
```

Terminal · Après la fusion : actualiser la principale

```
git switch main
```

Terminal · Commande à lancer au moment indiqué

```
git pull --ff-only
```

Tu sais que c'est bon quand : Tu as un pipeline rouge avec son lien et son motif, puis un pipeline vert sur le dernier commit, et la MR est fusionnée. Un rouge bloque la fusion ; un vert seul ne prouve pas qu'elle est autorisée.

Indice si tu bloques : La régression doit venir du code, jamais des tests : si tu modifies un test pour le faire passer, tu affaiblis la vérification. Le message de blocage et la règle de fusion sont à lire dans l'interface GitLab de la MR.

Avant de livrer

- Pour T2 à T5, ValueError doit être levée ; son absence est un échec, et toute autre exception est également un échec. Les six sous-cas T4 sont testés indépendamment.
- Le pipeline de MR teste le dernier commit, un compte habilité constate le blocage lors du rouge, puis la correction donne un dernier pipeline vert.

- Trois fonctions de contrôle GitLab sont vérifiées : MR ouverte, pipeline de MR, règle qui exige un pipeline réussi.

MR2 - feat/connexion (70 min)

Cette fonction sera appelée pour chaque port en MR3. Avance dans l'ordre des paliers et vérifie chaque résultat avant de poursuivre.

Avant de commencer

- Point de départ : MR1 fusionnée ; les cas T1 à T5 passent et le pipeline existe.
- Ce que tu vas produire : tester_port dans portpilot.py et le cas T6 dans verification.py : une connexion locale, quatre états possibles, socket fermée.
- Pourquoi : Cette fonction sera appelée pour chaque port en MR3.
- Commande des tests depuis la racine : `python3 verification.py`. Les cas ci-dessous sont à écrire par toi.

Palier 1 - Repartir de main à jour (10 min)

Créer la branche de la MR 2 depuis la branche principale qui contient déjà la MR 1 fusionnée.

Où tu travailles : Dépôt

Pour comprendre

- La branche principale (main) contient le travail déjà fusionné. Ta branche feat/... reçoit les changements de cette mission.
- Un dépôt « propre » n'a aucune modification en attente : git status te le confirme.

Actions dans l'ordre

1. Ouvre workspaces/portpilot dans VS Code, ou Coder dans Cyber Atelier avec cwd sur « . ».
2. Vérifie que MR1 est fusionnée.
3. Lis git status ; enregistre ton travail en cours avant de changer de branche.
4. Reviens sur main avec git switch main, mets-la à jour avec git pull --ff-only, puis crée feat/connexion. Remplace main par master si ton dépôt utilise ce nom.
5. Lance verification.py : T1 à T5 doivent passer sur cette base.
6. Vérifie le nom de ta branche avec git status. Si elle existe déjà, reprends-la avec git switch sans -c.

Terminal · Relire l'état du dépôt

Terminal du parcours : reste à la racine du dépôt. Exécute ces lignes au moment demandé dans les actions, avec les modifications manuelles entre les blocs.

```
git status
```

Terminal · Après la fusion : actualiser la principale

```
git switch main
```

Terminal · Commande à lancer au moment indiqué

```
git pull --ff-only
```

Terminal · Commande à lancer au moment indiqué

```
git switch -c feat/connexion
```

Terminal · Commande à lancer au moment indiqué

Terminal du parcours : reste à la racine du dépôt. Exécute ces lignes au moment demandé dans les actions, avec les modifications manuelles entre les blocs. Lance le programme et echo $? ensemble : le nombre affiché est le code du programme ; la console peut afficher code 0 pour echo lui-même.

```
python3 verification.py; echo $?
```

Tu sais que c'est bon quand : git status affiche « On branch feat/connexion », le dépôt est propre, et verification.py se termine avec le code 0.

Indice si tu bloques : Si ta branche principale n'a pas la fusion de la MR 1, git pull --ff-only ne peut pas le deviner : vérifie le graphe avec git log --oneline --graph.

Palier 2 - Socket d'écoute et port 0 : T6 rouge (10 min)

Comprendre une socket d'écoute locale, puis écrire T6 qui en ouvre une et appelle tester_port.

Où tu travailles : verification.py

Pour comprendre

- Une socket d'écoute attend une connexion. bind choisit l'adresse ; listen démarre l'écoute ; getsockname donne le port effectivement attribué.
- Le port 0 demande au système un port libre pour le test. Ce n'est pas un port à scanner.

Éditeur · Exemple Python à lire ou essayer

Éditeur : copie cet exemple dans exemple.py (jetable), enregistre-le, puis lance python3 exemple.py. Si plusieurs exemples sont proposés, essaie-les un par un en remplaçant le contenu du fichier. Retire ensuite les fichiers générés et le fichier d'exemple.

```
import socket

with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as ecoute:
    ecoute.bind(("127.0.0.1", 0))
    ecoute.listen()
    print("port attribué :", ecoute.getsockname()[1])
```

Cas de test à écrire

L'entrée est ce que tu donnes au code ; l'attendu est ce que ton test doit vérifier.

T6 - Connexion à une écoute éphémère

- Entrée : Le port attribué par le système à une écoute TCP locale encore ouverte
- Résultat attendu : tester_port retourne exactement OUVERT.

Actions dans l'ordre

1. Essaie l'exemple d'écoute locale et observe le port choisi.
2. Écris T6 dans verification.py : crée une socket IPv4, lie-la à 127.0.0.1 et au port 0, puis appelle listen.
3. Lis le port attribué et garde l'écoute ouverte pendant l'appel à tester_port. Vérifie le retour exact "OUVERT".
4. Assure la fermeture de l'écoute même si le test échoue.
5. Lance : l'absence de tester_port explique le rouge attendu.

Terminal · Commande à lancer au moment indiqué

Terminal du parcours : reste à la racine du dépôt. Exécute ces lignes au moment demandé dans les actions, avec les modifications manuelles entre les blocs.

```
python3 verification.py
```

Tu sais que c'est bon quand : verification.py échoue, et le message montre que tester_port n'est pas encore disponible.

Indice si tu bloques : Le getsockname() d'une socket liée au port 0 donne un couple (adresse, port) : le port est le second élément. Pas de accept, pas de thread, pas de time.sleep dans ce test.

Palier 3 - tester_port minimal : T6 vert (10 min)

Écrire tester_port avec l'hôte constant 127.0.0.1, le délai de 0,5 seconde et une seule tentative, puis faire passer T6.

Où tu travailles : portpilot.py ; verification.py

Pour comprendre

- socket.create_connection tente d'ouvrir une connexion vers un couple (hôte, port). Le timeout borne son attente.
- Une connexion réussie doit être fermée après son usage, même si elle ne sert qu'à vérifier l'ouverture.

Actions dans l'ordre

1. Consulte l'aide locale de socket.create_connection avec la commande proposée.
2. Ajoute tester_port(port) dans portpilot.py : hôte fixe 127.0.0.1, timeout 0,5 seconde et une seule tentative.
3. Pour ce palier, une connexion réussie retourne "OUVERT". Ferme la connexion créée.
4. Relance T1 à T6 puis commite. Les erreurs sont traitées au prochain palier.

Terminal · Commande à lancer au moment indiqué

Terminal du parcours : reste à la racine du dépôt. Exécute ces lignes au moment demandé dans les actions, avec les modifications manuelles entre les blocs.

```
python3 -c "import socket; help(socket.create_connection)"
```

Terminal · Commande à lancer au moment indiqué

Terminal du parcours : reste à la racine du dépôt. Exécute ces lignes au moment demandé dans les actions, avec les modifications manuelles entre les blocs. Lance le programme et echo $? ensemble : le nombre affiché est le code du programme ; la console peut afficher code 0 pour echo lui-même.

```
python3 verification.py; echo $?
```

Terminal · Enregistrer le travail vérifié

Terminal du parcours : reste à la racine du dépôt. Exécute ces lignes au moment demandé dans les actions, avec les modifications manuelles entre les blocs.

```
git add portpilot.py verification.py
git commit -m "Ajoute tester_port et le test T6"
```

Tu sais que c'est bon quand : T1 à T6 passent et git log --oneline montre ton commit « Ajoute tester_port et le test T6 ».

Indice si tu bloques : Une connexion qui réussit ne signifie que l'ouverture TCP. Ne lis pas d'autre donnée sur la socket : aucune donnée applicative n'est envoyée par PortPilot.

Palier 4 - Les autres états : REFUSE, TIMEOUT, ERREUR (10 min)

Classer les exceptions réseau dans le bon état, avec le bon ordre des except.

Où tu travailles : portpilot.py

Pour comprendre

- Les exceptions forment une hiérarchie : ConnectionRefusedError est un cas particulier d'OSError. Un except trop général placé en premier masque les cas précis.
- Les états sont des chaînes qui décrivent ce qui s'est passé pendant la tentative.

Éditeur · Exemple Python à lire ou essayer

Éditeur : copie cet exemple dans exemple.py (jetable), enregistre-le, puis lance python3 exemple.py. Si plusieurs exemples sont proposés, essaie-les un par un en remplaçant le contenu du fichier. Retire ensuite les fichiers générés et le fichier d'exemple.

```
try:
    with open("fichier_inexistant.txt") as f:
        print(f.read())
except FileNotFoundError:
    print("fichier absent : exception précise")
except OSError:
    print("autre erreur du système")
```

Règles à respecter

- Connexion réussie -> OUVERT.
- ConnectionRefusedError -> REFUSE.
- Dépassement du délai -> TIMEOUT.
- Autre OSError -> ERREUR.

Actions dans l'ordre

1. Essaie l'exemple FileNotFoundError/OSError et repère l'ordre des except.
2. Ajoute les traitements dans tester_port, avec les exceptions précises avant OSError.
3. Relis chaque branche : elle retourne un seul des quatre états du contrat.
4. Relance T1 à T6. T6 vérifie le succès ; relis aussi le traitement des trois autres états.

Terminal · Commande à lancer au moment indiqué

Terminal du parcours : reste à la racine du dépôt. Exécute ces lignes au moment demandé dans les actions, avec les modifications manuelles entre les blocs. Lance le programme et echo $? ensemble : le nombre affiché est le code du programme ; la console peut afficher code 0 pour echo lui-même.

```
python3 verification.py; echo $?
```

Terminal · Enregistrer le travail vérifié

Terminal du parcours : reste à la racine du dépôt. Exécute ces lignes au moment demandé dans les actions, avec les modifications manuelles entre les blocs.

```
git add portpilot.py
git commit -m "Classe les erreurs de connexion en REFUSE, TIMEOUT et ERREUR"
```

Tu sais que c'est bon quand : tester_port peut retourner les quatre états, dans cet ordre de traitement, et T1 à T6 passent toujours.

Indice si tu bloques : Compte les exceptions de la plus précise à la plus générale. Pour TIMEOUT, vérifie dans l'aide de Python quelle exception correspond au dépassement du délai dans ta version de Python.

Palier 5 - Fermer la socket, même en cas d'erreur (10 min)

Garantir la fermeture de toute socket créée, même quand une exception survient.

Où tu travailles : portpilot.py

Pour comprendre

- Une socket reste une ressource ouverte tant qu'elle n'est pas fermée. with la ferme à la sortie du bloc, y compris si une exception survient.
- finally sert aussi à effectuer un nettoyage après un succès ou une erreur.

Éditeur · Exemple Python à lire ou essayer

Éditeur : copie cet exemple dans exemple.py (jetable), enregistre-le, puis lance python3 exemple.py. Si plusieurs exemples sont proposés, essaie-les un par un en remplaçant le contenu du fichier. Retire ensuite les fichiers générés et le fichier d'exemple.

```
def diviser(a, b):
    try:
        return a / b
    finally:
        print("nettoyage fait")

print(diviser(6, 3))
diviser(1, 0)
```

Actions dans l'ordre

1. Essaie l'exemple finally avec une division valide, puis une division par zéro. Observe le nettoyage dans les deux cas.
2. Relis tous les chemins de tester_port : chaque socket créée doit être fermée.
3. Complète le nettoyage si nécessaire, sans modifier T6.
4. Relance verification.py et vérifie que la suite reste verte.

Terminal · Commande à lancer au moment indiqué

Terminal du parcours : reste à la racine du dépôt. Exécute ces lignes au moment demandé dans les actions, avec les modifications manuelles entre les blocs. Lance le programme et echo $? ensemble : le nombre affiché est le code du programme ; la console peut afficher code 0 pour echo lui-même.

```
python3 verification.py; echo $?
```

Tu sais que c'est bon quand : Dans tester_port, chaque socket créée est fermée dans tous les cas, et verification.py passe toujours.

Indice si tu bloques : Un bloc with n'a pas besoin de close() explicite. Si tu crées une socket en dehors d'un with, pense à finally pour la fermer.

Palier 6 - Relecture, commit et pipeline de MR (10 min)

Livrer la connexion et son test dans une MR au pipeline vert.

Où tu travailles : Branche feat/connexion ; MR GitLab

Pour comprendre

- git diff montre les lignes modifiées ; git diff --staged montre celles choisies pour le prochain commit. Le diff de la MR compare toute ta branche à main.
- La demande de fusion (MR dans GitLab, PR dans GitHub) permet de relire le travail avant de l'intégrer à main.

Actions dans l'ordre

1. Lance la suite complète avec la commande indiquée au début de la mission.
2. Relis T6 : écoute éphémère locale, sans port fixe, serveur externe, thread, accept ou attente artificielle. Le pipeline doit exécuter T1 à T6 sans laboratoire.
3. Relis git status et git diff : la livraison doit contenir portpilot.py et verification.py, avec les modifications nécessaires de cette mission.

4. Ajoute les fichiers par leur nom, relis git diff --staged, puis commite ce qui ne l’est pas encore.
5. Pousse feat/connexion et ouvre la demande de fusion vers main.
6. Dans GitLab, ouvre le pipeline du dernier commit : vérifie qu’il a réellement exécuté les tests et qu’il est vert. Un test local réussi ne le remplace pas.
7. Relis le diff dans GitLab. La fusion et le retour sur main se font au palier suivant.

Terminal · Relire l’état du dépôt

Terminal du parcours : reste à la racine du dépôt. Exécute ces lignes au moment demandé dans les actions, avec les modifications manuelles entre les blocs.

```
git status
git diff
```

Terminal · Enregistrer le travail vérifié

```
git add portpilot.py verification.py
git commit -m "Termine la connexion locale et relit les états d’erreur"
```

Terminal · Publier la branche ou ouvrir la demande de fusion

```
git push -u origin feat/connexion
```

Terminal · Relire l’état du dépôt

```
git log --oneline -3
```

Tu sais que c’est bon quand : La MR 2 existe, son pipeline sur le dernier commit est vert et il exécute T1 à T6.

Indice si tu bloques : Un pipeline vert sur un commit précédent ne suffit pas : c’est le dernier commit de la MR qui doit être testé. Si la CI ne lance pas T6, regarde la commande de ton job.

Palier 7 - Fusion et retour sur main (10 min)

Fusionner la MR 2 après un pipeline vert, puis repartir d’une branche principale à jour pour la MR 3.

Où tu travailles : MR GitLab ; branche principale

Pour comprendre

- Le pipeline vert est une condition de fusion de l’école ; ta branche principale locale doit ensuite récupérer cette fusion.
- La prochaine mission repart de cette version commune.

Actions dans l’ordre

1. Si tu n’as pas encore fusionné MR2, relis son diff et son dernier pipeline, puis fusionne lorsque la règle de l’école l’autorise.
2. Reviens sur main et lance git pull --ff-only.
3. Vérifie dans git log que la fusion est présente, puis relance T1 à T6.
4. Ne crée la branche de MR3 qu’après ces vérifications.

Terminal · Après la fusion : actualiser la principale

Terminal du parcours : reste à la racine du dépôt. Exécute ces lignes au moment demandé dans les actions, avec les modifications manuelles entre les blocs.

```
git switch main
```

Terminal · Commande à lancer au moment indiqué

```
git pull --ff-only
```

Terminal · Relire l’état du dépôt

```
git log --oneline -5
```

Tu sais que c’est bon quand : La MR 2 est fusionnée après pipeline vert du dernier commit, et git log --oneline sur main montre le commit de fusion.

Indice si tu bloques : Si GitLab refuse la fusion, lis le motif affiché dans la MR : il s’agit souvent d’un pipeline pas encore réussi ou d’une règle de fusion.

Avant de livrer

- T1 à T6 restent appelés et passent localement et dans un runner ordinaire.
- Le test n’emploie aucun port fixe, serveur externe, thread, accept ou attente artificielle.
- La socket d’écoute est fermée dans tous les cas.

MR3 - feat/scan-bilan (80 min)

Tu assembles validation, connexion et bilan sans élargir le périmètre du scanner. Avance dans l’ordre des paliers et vérifie chaque résultat avant de poursuivre.

Avant de commencer

- Point de départ : MR2 fusionnée ; T1 à T6 passent sur la branche principale.
- Ce que tu vas produire : scanner_ports, resumer et main ; T7 et T8 ; README et livraison ZIP par la CI.
- Pourquoi : Tu assembles validation, connexion et bilan sans élargir le périmètre du scanner.
- Commande des tests depuis la racine : `python3 verification.py`. Les cas ci-dessous sont à écrire par toi.

Palier 1 - Branche à jour et liste des livrables (10 min)

Partir de la branche principale à jour et savoir exactement ce que la MR 3 doit livrer.

Où tu travailles : Dépôt

Pour comprendre

- La branche principale (main) contient le travail déjà fusionné. Ta branche feat/... reçoit les changements de cette mission.
- Un dépôt « propre » n’a aucune modification en attente : git status te le confirme.

Actions dans l’ordre

1. Ouvre workspaces/portpilot dans VS Code, ou Coder dans Cyber Atelier avec cwd sur « . ».
2. Vérifie que MR2 est fusionnée.
3. Lis git status ; enregistre ton travail en cours avant de changer de branche.
4. Reviens sur main avec git switch main, mets-la à jour avec git pull --ff-only, puis crée feat/scan-bilan. Remplace main par master si ton dépôt utilise ce nom.
5. Lance T1 à T6. Relis les livrables : portpilot.py, verification.py, README.md, .gitignore et .gitlab-ci.yml ; le ZIP final ne contient que les trois premiers.
6. Vérifie le nom de ta branche avec git status. Si elle existe déjà, reprends-la avec git switch sans -c.

Terminal · Relire l’état du dépôt

Terminal du parcours : reste à la racine du dépôt. Exécute ces lignes au moment demandé dans les actions, avec les modifications manuelles entre les blocs.

```
git status
```

Terminal · Après la fusion : actualiser la principale

```
git switch main
```

Terminal · Commande à lancer au moment indiqué

```
git pull --ff-only
```

Terminal · Commande à lancer au moment indiqué

```
git switch -c feat/scan-bilan
```

Terminal · Commande à lancer au moment indiqué

Terminal du parcours : reste à la racine du dépôt. Exécute ces lignes au moment demandé dans les actions, avec les modifications manuelles entre les blocs. Lance le programme et echo $? ensemble : le nombre affiché est le code du programme ; la console peut afficher code 0 pour echo lui-même.

```
python3 verification.py; echo $?
```

Tu sais que c'est bon quand : git status affiche « On branch feat/scan-bilan », le dépôt est propre, et verification.py se termine avec le code 0.

Indice si tu bloques : Si T6 échoue sur la branche principale, ne continue pas : la MR 2 n'est pas stable. Compare avec git log --oneline.

Palier 2 - Valider toute la liste d'abord : T7 rouge (10 min)

Comprendre pourquoi on valide l'ensemble avant d'agir, puis écrire T7 qui doit échouer.

Où tu travailles : verification.py

Pour comprendre

- Valider toute la liste avant d'agir empêche des connexions partielles si un port plus loin dans la liste est invalide.
- T7 ouvre lui-même ses deux écoutes locales et les garde disponibles pendant tout le scan.

Éditeur · Exemple Python à lire ou essayer

Éditeur : copie cet exemple dans exemple.py (jetable), enregistre-le, puis lance python3 exemple.py. Si plusieurs exemples sont proposés, essaie-les un par un en remplaçant le contenu du fichier. Retire ensuite les fichiers générés et le fichier d'exemple.

```
articles = ["pain", "", "lait"]

for article in articles:
    if article == "":
        print("liste refusée : article vide")
        break
else:
    print("liste valide, on commence les achats")
```

Cas de test à écrire

L'entrée est ce que tu donnes au code ; l'attendu est ce que ton test doit vérifier.

T7 - Scan de deux écoutes dans l'ordre

- Entrée : Deux sockets éphémères ouvertes, fournies dans un ordre choisi
- Résultat attendu : Deux paires port/OUVERT exactement dans l'ordre de l'entrée.

Actions dans l'ordre

1. Observe l'exemple du panier refusé avant le début des achats.
2. Écris T7 : crée deux sockets d'écoute IPv4 avec le port 0, puis lis leurs deux ports distincts.

3. Donne ces ports à scanner_ports dans un ordre choisi. Vérifie les deux paires [port, "OUVERT"] dans exactement cet ordre.
4. Ferme les deux écoutes systématiquement après l'essai, même si une assertion échoue.
5. Lance le test et constate le rouge lié à l'absence de scanner_ports.

Terminal · Commande à lancer au moment indiqué

Terminal du parcours : reste à la racine du dépôt. Exécute ces lignes au moment demandé dans les actions, avec les modifications manuelles entre les blocs.

```
python3 verification.py
```

Tu sais que c'est bon quand : T7 est écrit, il échoue, et tu peux dire pourquoi.

Indice si tu bloques : Le test doit garder les deux écoutes ouvertes tant que scanner_ports n'a pas fini. Un with imbriqué ou un try/finally peut garantir la fermeture des deux sockets.

Palier 3 - scanner_ports : T7 vert et ordre conservé (10 min)

Écrire scanner_ports qui valide la liste entière, fait une tentative par port dans l'ordre reçu, puis faire passer T7.

Où tu travailles : portpilot.py ; verification.py

Pour comprendre

- Une paire [port, état] relie la valeur testée à son résultat. Une liste de paires garde l'ordre de tes tentatives.
- Réutiliser valider_ports puis tester_port conserve les règles déjà vérifiées.

Éditeur · Exemple Python à lire ou essayer

Éditeur : copie cet exemple dans exemple.py (jetable), enregistre-le, puis lance python3 exemple.py. Si plusieurs exemples sont proposés, essaie-les un par un en remplaçant le contenu du fichier. Retire ensuite les fichiers générés et le fichier d'exemple.

```
fruits = ["pomme", "poire"]
resultats = []
for position, fruit in enumerate(fruits):
    resultats.append([position, fruit])
print(resultats)
```

Exemple de résultat attendu

Ce que tu donnes :

```
Deux ports locaux ouverts, fournis dans l'ordre 53002 puis 53001.
```

Ce que tu dois obtenir :

```
[[53002, "OUVERT"], [53001, "OUVERT"]]
```

Ces nombres illustrent la forme ; T7 utilise les ports réellement attribués par le système.

Actions dans l'ordre

1. Crée scanner_ports(ports) : valide toute la liste avant le premier appel réseau.
2. Parcours les ports dans leur ordre ; appelle tester_port une fois par port et conserve [port, état].
3. Retourne la liste obtenue. Une liste refusée ne doit lancer aucune tentative.
4. Relance T1 à T7, puis commite.

Terminal · Commande à lancer au moment indiqué

Terminal du parcours : reste à la racine du dépôt. Exécute ces lignes au moment demandé dans les actions, avec les modifications manuelles entre les blocs. Lance le programme et echo $? ensemble : le nombre

affiché est le code du programme ; la console peut afficher code 0 pour echo lui-même.

```
python3 verification.py; echo $?
```

Terminal · Enregistrer le travail vérifié

Terminal du parcours : reste à la racine du dépôt. Exécute ces lignes au moment demandé dans les actions, avec les modifications manuelles entre les blocs.

```
git add portpilot.py verification.py
git commit -m "Ajoute scanner_ports et le test T7"
```

Tu sais que c'est bon quand : T1 à T7 passent et git log --oneline montre ton commit « Ajoute scanner_ports et le test T7 ».

Indice si tu bloques : scanner_ports ne doit pas réimplémenter la connexion : il réutilise tester_port. Un port invalide dans la liste doit faire échouer la validation avant la boucle de connexion.

Palier 4 - resumer et T8 : le bilan (10 min)

Compter les états dans un dictionnaire à cinq compteurs, et écrire T8 pour vérifier le comptage.

Où tu travailles : portpilot.py ; verification.py

Pour comprendre

- Le bilan compte des états déjà connus ; il ne se connecte à rien. Tu peux donc le tester avec des états fabriqués.
- Une catégorie vide a tout de même un compteur zéro.

Éditeur · Exemple Python à lire ou essayer

Éditeur : copie cet exemple dans exemple.py (jetable), enregistre-le, puis lance python3 exemple.py. Si plusieurs exemples sont proposés, essaie-les un par un en remplaçant le contenu du fichier. Retire ensuite les fichiers générés et le fichier d'exemple.

```
fruits = ["pomme", "poire", "pomme"]
compte = {}
for fruit in fruits:
    compte[fruit] = compte.get(fruit, 0) + 1
print(compte)
```

Exemple de résultat attendu

Ce que tu donnes :

```
Deux OUVERT, un REFUSE, un TIMEOUT, un ERREUR.
```

Ce que tu dois obtenir :

```
{"total": 5, "ouverts": 2, "refuses": 1, "timeouts": 1, "erreurs": 1}
```

T8 vérifie ce comptage ; il ne provoque pas les trois erreurs réseau.

Cas de test à écrire

L'entrée est ce que tu donnes au code ; l'attendu est ce que ton test doit vérifier.

T8 - Bilan mixte et vide

- Entrée : Deux OUVERT, un REFUSE, un TIMEOUT, un ERREUR ; puis liste vide
- Résultat attendu : total 5, ouverts 2, refuses 1, timeouts 1, erreurs 1 ; cinq zéros pour la liste vide.

Actions dans l'ordre

1. Écris T8 : deux OUVERT, un REFUSE, un TIMEOUT et un ERREUR. Compare les cinq compteurs.

2. Ajoute le cas de la liste vide : les cinq compteurs valent zéro.
3. Crée resumer(resultats), retourne un dictionnaire et vérifie que les quatre catégories totalisent total.
4. Relance T1 à T8 et commite.

Terminal · Commande à lancer au moment indiqué

Terminal du parcours : reste à la racine du dépôt. Exécute ces lignes au moment demandé dans les actions, avec les modifications manuelles entre les blocs. Lance le programme et echo $? ensemble : le nombre affiché est le code du programme ; la console peut afficher code 0 pour echo lui-même.

```
python3 verification.py; echo $?
```

Terminal · Enregistrer le travail vérifié

Terminal du parcours : reste à la racine du dépôt. Exécute ces lignes au moment demandé dans les actions, avec les modifications manuelles entre les blocs.

```
git add portpilot.py verification.py
git commit -m "Ajoute resumer et le test T8"
```

Tu sais que c'est bon quand : T1 à T8 passent, et tu peux expliquer pourquoi T8 ne prouve pas que les erreurs réseau se déclenchent.

Indice si tu bloques : Commence par le cas de la liste vide : un dictionnaire ou des compteurs à zéro suffisent. Vérifie que la somme des catégories égale total dans ton test.

Palier 5 - main : message clair et import sans lancement (10 min)

Séparer le point d'entrée des fonctions, pour que l'import du module ne lance jamais le scan.

Où tu travailles : portpilot.py

Pour comprendre

- La garde __main__ empêche le lancement automatique du scan lors de l'import depuis verification.py.
- main rassemble configuration, scan et affichage ; les calculs restent dans leurs fonctions.

Éditeur · Exemple Python à lire ou essayer

Éditeur : copie cet exemple dans exemple.py (jetable), enregistre-le, puis lance python3 exemple.py. Si plusieurs exemples sont proposés, essaie-les un par un en remplaçant le contenu du fichier. Retire ensuite les fichiers générés et le fichier d'exemple.

```
def saluer():
    print("Bonjour")

if __name__ == "__main__":
    saluer()
```

Actions dans l'ordre

1. Essaie l'exemple dans exemple.py : lancement direct -> Bonjour ; import -> aucune sortie. Supprime le fichier d'essai.
2. Écris main pour utiliser la liste configurée, afficher un résultat par port puis le bilan.
3. Traite une liste refusée avec un message clair plutôt qu'une trace brute.
4. Place le lancement sous la garde __main__, vérifie un import silencieux et relance la suite.
5. Commite les changements.

Terminal · Commande à lancer au moment indiqué

Terminal du parcours : reste à la racine du dépôt. Exécute ces lignes au moment demandé dans les actions, avec les modifications manuelles entre les blocs.

```
python3 -c "import portpilot"
```

Terminal · Commande à lancer au moment indiqué

Terminal du parcours : reste à la racine du dépôt. Exécute ces lignes au moment demandé dans les actions, avec les modifications manuelles entre les blocs. Lance le programme et echo $? ensemble : le nombre affiché est le code du programme ; la console peut afficher code 0 pour echo lui-même.

```
python3 verification.py; echo $?
```

Terminal · Enregistrer le travail vérifié

Terminal du parcours : reste à la racine du dépôt. Exécute ces lignes au moment demandé dans les actions, avec les modifications manuelles entre les blocs.

```
git add portpilot.py verification.py
git commit -m "Ajoute main et le message de liste invalide"
```

Tu sais que c'est bon quand : L'import de portpilot n'affiche rien, un lancement direct présente le bilan ou un message clair, et verification.py passe.

Indice si tu bloques : Pour le message clair, attrape ValueError autour de la validation de la liste dans main et affiche le texte de l'erreur. Le fichier principal ne doit pas planter avec une trace inutile pour une liste invalide.

Palier 6 - README final avant la CI (10 min)

Décrire le projet et ses limites avant que la CI de livraison ne construise l'artefact.

Où tu travailles : README.md

Pour comprendre

- Le README explique comment configurer, lancer et interpréter ton programme. Il doit distinguer ce que le scan observe et ce que les tests prouvent.
- Un port OUVERT indique une connexion possible, sans identifier une vulnérabilité ou la nature du service.

Actions dans l'ordre

1. Documente objectif, limites, prérequis, liste configurée, lancement, quatre états, tests et téléchargement de l'artefact.
2. Indique le périmètre : 127.0.0.1, scan séquentiel, une tentative et 0,5 seconde par port.
3. Explique que T8 compte des états fabriqués et ne déclenche pas REFUSE, TIMEOUT ou ERREUR sur le réseau.
4. Décris la démonstration du palier 8 avec le labo sur 8000, puis commite le README.

Terminal · Relire l'état du dépôt

Terminal du parcours : reste à la racine du dépôt. Exécute ces lignes au moment demandé dans les actions, avec les modifications manuelles entre les blocs.

```
git status
```

Terminal · Enregistrer le travail vérifié

```
git add README.md
git commit -m "Finalise le README de PortPilot"
```

Tu sais que c'est bon quand : Le README contient les sections demandées, il est cohérent avec ce que fait le code et il est commité.

Indice si tu bloques : Un README qui promet trop est une erreur : relis les limites dans le guide du parcours et écris-les avec tes mots. Un test local ou un pipeline vert ne prouve pas la sécurité du scan.

Palier 7 - CI en deux stages et artefact ZIP (10 min)

Ajouter une étape de livraison qui ne tourne qu'après les tests et produit un ZIP contrôlé.

Où tu travailles : .gitlab-ci.yml ; portpilot.zip

Pour comprendre

- Un stage regroupe des jobs. L'ordre test puis livraison empêche de livrer une version dont les tests échouent.
- Un artefact est un fichier produit par le pipeline et conservé pour téléchargement.

Éditeur · Exemple Python à lire ou essayer

Éditeur : copie cet exemple dans exemple.py (jetable), enregistre-le, puis lance python3 exemple.py. Si plusieurs exemples sont proposés, essaie-les un par un en remplaçant le contenu du fichier. Retire ensuite les fichiers générés et le fichier d'exemple.

```
import zipfile

with open("notes.txt", "w", encoding="utf-8") as f:
    f.write("exemple\n")
with zipfile.ZipFile("exemple.zip", "w") as archive:
    archive.write("notes.txt")
```

Actions dans l'ordre

1. Essaie l'exemple zipfile, liste son contenu puis retire ses fichiers d'essai.
2. Adapte la CI à deux stages : test, puis livraison après réussite des tests.
3. Crée portpilot.zip avec exactement portpilot.py, verification.py et README.md, sans dossier supplémentaire ni cache.
4. Vérifie localement la liste des fichiers du ZIP ; configure sa conservation comme artefact pendant sept jours.
5. Commite la CI, pousse et ouvre la MR si nécessaire. Vérifie les deux stages du pipeline du dernier commit.

Terminal · Commande à lancer au moment indiqué

Terminal du parcours : reste à la racine du dépôt. Exécute ces lignes au moment demandé dans les actions, avec les modifications manuelles entre les blocs.

```
python3 -m zipfile -l exemple.zip
```

Terminal · Enregistrer le travail vérifié

```
git add .gitlab-ci.yml
git commit -m "Ajoute les stages test et livraison de PortPilot"
```

Terminal · Publier la branche ou ouvrir la demande de fusion

```
git push
```

Terminal · Relire l'état du dépôt

```
git log --oneline -1
```

Tu sais que c'est bon quand : Le pipeline du dernier commit affiche le stage test vert, puis la livraison, et l'artefact apparaît dans la page du pipeline.

Indice si tu bloques : Les règles de livraison ne doivent pas contourner le stage test : la livraison suit les tests. Supprime les fichiers jetables de l'exemple (notes.txt, exemple.zip) avant de commiter.

Palier 8 - Démonstration locale, artefact et fusion (10 min)

Constater le comportement du scan sur le labo, vérifier l'artefact du dernier pipeline vert, puis fusionner la MR 3.

Où tu travailles : MR GitLab

Pour comprendre

- La démonstration observe le laboratoire réel. La suite de tests continue à créer ses propres écoutes : elle ne dépend pas de ce labo.
- L'artefact à relire est celui du dernier pipeline vert ; il expire au bout de sept jours.

Actions dans l'ordre

1. Sur le Mac, lance python3 labs.py depuis cyberatelier dans un premier terminal. Dans Docker, utilise le bouton du labo à la place.
2. Dans le même environnement, configure 8000 dans la liste de PortPilot et lance le programme : attends OUVERT.
3. Arrête le labo avec Ctrl+C ou son bouton, relance : attends REFUSE. Ne démarre pas un autre serveur sur ce port.
4. Si tu as modifié la liste livrée, commite et pousse cette configuration, puis attends son pipeline.
5. Télécharge le ZIP du dernier pipeline vert et vérifie ses trois fichiers.
6. Relis la MR, fusionne lorsque les règles le permettent, puis mets main à jour.

Terminal · Option Mac : démarrer le laboratoire dans un autre terminal

Terminal séparé sur ton Mac, depuis le dossier cyberatelier. Laisse-le ouvert pendant la démonstration, puis arrête avec Ctrl+C. Dans Docker, utilise plutôt le bouton du laboratoire.

```
python3 labs.py
```

Terminal · Commande à lancer au moment indiqué

Terminal du parcours : reste à la racine du dépôt. Exécute ces lignes au moment demandé dans les actions, avec les modifications manuelles entre les blocs.

```
python3 portpilot.py
```

Terminal · Commande à lancer au moment indiqué

```
python3 -m zipfile -l portpilot.zip
```

Terminal · Après la fusion : actualiser la principale

```
git switch main
```

Terminal · Commande à lancer au moment indiqué

```
git pull --ff-only
```

Terminal · Relire l'état du dépôt

```
git log --oneline -5
```

Tu sais que c'est bon quand : OUVERT pour 8000 pendant labs.py, REFUSE après Ctrl+C, l'artefact contient exactement trois fichiers, et la MR 3 est fusionnée après pipeline vert.

Indice si tu bloques : Le succès du script local ne décide pas si GitLab autorise une fusion ; vérifie aussi le commit et la règle de fusion. Garde le lien du pipeline rouge de la MR 1 comme preuve.

Avant de livrer

- L'import du module n'exécute pas main ; une liste invalide affiche un message clair.

- Les huit tests sont appelés ; T4 vérifie ses six sous-cas séparément. Les tests ne prétendent pas couvrir le déclenchement des trois états d’erreur réseau.
- portpilot.zip contient exactement portpilot.py, verification.py et README.md, généré après succès des tests et conservé sept jours.
- Trois MR sont fusionnées au pipeline vert du dernier commit ; la preuve du rouge MR1 est conservée.

Cycle de travail et validation

Pour chaque MR, développe et teste localement, lis git status et git diff, ajoute explicitement les fichiers nécessaires, crée des commits descriptifs et pousse sur la branche de la MR. Ouvre la MR vers la branche principale, vérifie le pipeline du dernier commit et relis le diff. Après chaque fusion, actualise la branche principale avant de créer la suivante. Une seule MR finale ne remplace pas les trois étapes.

Commande locale depuis la racine : python verification.py. Adapte python en python3 si nécessaire. Une assertion échouée doit rester visible et rendre le processus non nul. Ne lance pas Python en mode optimisé. Les huit tests couvrent validation valide, liste vide, trop d’éléments, six cas invalides testés séparément dans des listes à un élément, doublons, connexion ouverte, scan de deux ports dans l’ordre et bilan mixte/vide.

Livrables et limites

Le dépôt final contient portpilot.py, verification.py, README.md, .gitignore et .gitlab-ci.yml, trois MR fusionnées au dernier pipeline vert, le lien du pipeline rouge de MR1 avec son motif, et l’artefact portpilot.zip du pipeline vert de MR3. Le ZIP contient uniquement les trois fichiers annoncés.

L’outil tente seulement une connexion TCP sur 127.0.0.1, de façon séquentielle, avec une tentative et un délai de 0,5 seconde par port. Il ne prouve ni vulnérabilité, ni disponibilité générale, ni identité d’un service. Le projet ne couvre pas toutes les branches d’erreur réseau par tests automatisés. L’exercice concerne exclusivement un laboratoire local autorisé.